
Računarske arhitekture

PREDAVANJE 6 – IZUZECI

A solid blue horizontal bar at the bottom of the slide.

Teme za danas

1. Izuzeci (exceptions)
2. Višeciklusne operacije
3. Reorder buffer
4. History buffer
5. Future register file

Problem
Algoritam
Programski jezik
Sistemski softver
SW/HW Interfejs (ISA)
Mikroarhitektura
Digitalna logika
Uređaji
Fizikalni nivo

Mikroarhitektura je unutrašnja organizacija procesora koja određuje kako se instrukcije zadane ISA-om stvarno izvršavaju u hardveru.

Izuzetak (exception)

- **Izuzetak** je događaj koji prekida normalan tok izvršavanja programa.
- Nastaje kada procesor ili sistem detektuje posebnu situaciju.
- Nakon detekcije, kontrola toka instrukcija se preusmjerava na **rutinu za obradu**.
 - Obrada može omogućiti nastavak izvršavanja ili dovesti do prekida programa.
- Mnoge arhitekture i autori koriste stariji naziv **prekid**. Na primjer, x86 koristi termin prekid.
 - Mi slijedimo MIPS konvenciju i koristimo termin **izuzetak** za svaku neočekivanu promjenu toka **upravljanja**, ne praveći razliku da li je uzrok unutrašnji ili spoljašnji; **termin prekid** koristimo samo kada je događaj izazvan spolja.
- **Izuzeci su neophodan mehanizam za ispravno, sigurno i efikasno funkcionisanje računarskog sistema.**

Izuzeci

Zašto su važni?

- Omogućavaju detekciju i obradu grešaka
- Omogućavaju komunikaciju hardvera i operativnog sistema
- Podržavaju sistemske pozive i rad operativnog sistema
- Pomažu zaštitu memorije i resursa
- Omogućavaju prekidanje izvršavanja kada je potrebna hitna reakcija

Dijeljenje nulom

Nelegalna instrukcija

Prekoračenje u aritmetičkoj operaciji

Greška stranice pri pristupu stranici koja trenutno nije u glavnoj memoriji

Prekid sa tastature kada korisnik pritisne tipku

Prekid od diska kada je završeno čitanje ili upis

Prekid od mrežne kartice kada stigne mrežni paket

Syscall za ispis na ekran

Syscall za kreiranje ili završetak procesa

Pristup zabranjenoj memorijskoj adresi

Upis u memorijsku oblast samo za čitanje

Izvršavanje koda iz nedozvoljene memorijske oblasti

Greška pri pristupu kernel memoriji iz korisničkog režima

Tajmerski prekid za prebacivanje na drugi proces

Watchdog signal kada sistem ne reaguje ispravno

Upozorenje na problem sa napajanjem ili kritičan kvar uređaja

Izuzeci

Šta se dešava kada nastane izuzetak?

1. Procesor detektuje problem ili događaj
2. Privremeno prekida normalno izvršavanje instrukcija
3. Pamti potrebne informacije o trenutnom stanju
4. Prelazi na rutinu za obradu izuzetka
5. Nakon obrade:
 - a) izvršavanje se može nastaviti
 - b) ili program/sistem se mora zaustaviti

Osnovne podjele izuzetaka:

- softverski / hardverski
- sinhroni / asinhroni
- traženi / prinudni
- prekidi koji se mogu / ne mogu maskirati
- unutar / između instrukcija
- nastavljivi / završni
- precizni / neprecizni

Sinhroni i asinhroni izuzeci

Sinhroni izuzeci

- Nastaju direktno zbog izvršavanja konkretne instrukcije
- Može se tačno odrediti koja ih je instrukcija izazvala
- Primjeri:
 - a) Dijeljenje nulom
 - b) Nelegalna instrukcija
 - c) Greška stranice pri load/store
 - d) Overflow

Asinhroni izuzeci

- Ne nastaju direktno zbog tekuće instrukcije
- Dolaze spolja ili nezavisno od toka programa
- Primjeri:
 - a) Prekid sa tastature
 - b) Tajmerski prekid
 - c) Prekid od mrežne kartice

Traženi i prinudni izuzeci

Traženi izuzeci

- Program ih namjerno izaziva
- Koriste se za zahtjev prema operativnom sistemu ili za kontrolisanu obradu
- Primjeri:
 - a) Syscall
 - b) Trap instrukcija
 - c) Breakpoint

Prinudni izuzeci

- Program ih nije tražio
- Nastaju zbog greške ili nedozvoljenog stanja
- Primjeri:
 - a) Dijeljenje nulom
 - b) Pristup zabranjenoj memoriji
 - c) Nelegalna instrukcija

Prekidi koji se mogu i ne mogu maskirati

Maskabilni prekidi

- Mogu se privremeno zabraniti ili odgoditi
- Dolaze od standardnih periferijskih uređaja
- Primjeri:
 - a) Prekid sa tastature
 - b) Prekid od diska
 - c) Prekid od mrežne kartice

Nemaskabilni prekidi

- Ne mogu se ignorisati
- Označavaju vrlo ozbiljan događaj
- Primjeri:
 - a) Kritična hardverska greška
 - b) Ozbiljna greška memorije
 - c) Watchdog signal
 - d) Ozbiljan problem sa napajanjem

Nastavljivi i završni izuzeci

Nastavljivi izuzeci

- Nakon obrade, izvršavanje se može nastaviti
- Većinom se ista instrukcija ponovo izvrši
- Primjeri:
 - a) Greška stranice
 - b) Breakpoint nakon debugovanja
 - c) Sistemski poziv

Završni izuzeci

- Nakon obrade, program se ne može normalno nastaviti
- Primjeri:
 - a) Nelegalna instrukcija
 - b) Teška zaštitna greška
 - c) Fatalna hardverska greška

Precizni i neprecizni izuzeci

Precizni izuzeci

- Izuzetak je precizan ako u trenutku obrade važi:

1. sve starije instrukcije su potpuno završene
2. instrukcija koja je izazvala izuzetak je tačno poznata
3. sve mlađe instrukcije još nisu promijenile arhitekturno stanje

```
40 sub $11, $2, $4
44 and $12, $2, $5
48 or  $13, $2, $6
4C add $1,  $2, $1
50 slt $15, $6, $7
```

Sistem izgleda kao da se izvršavanje odvijalo strogo redom, i kao da je procesor stao tačno na instrukciji koja je izazvala izuzetak.

Neprecizni izuzeci

- Izuzetak je neprecizan ako u trenutku prijave nije jasno i uredno odvojeno:
1. koje starije instrukcije su završene
 2. koja je tačno instrukcija izazvala problem
 3. da li su neke mlađe instrukcije već promijenile stanje

Dakle, procesor ne izgleda kao da je jednostavno stao “na jednoj tački” programskog toka.

Zašto želimo precizne izuzetke?

- Semantika von-Neumannovog modela, ISA to zahtijeva - program se posmatra kao **sekvenca instrukcija koja se izvršava redom**
- Pomažu pri debugovanju softvera (debager jasno vidi koja instrukcija je izazvala problem, u kakvom su stanju registri i memorija, koje su instrukcije završene a koje nisu)
- Omogućavaju (jednostavan) oporavak od izuzetaka
- Omogućavaju (jednostavno) ponovno pokretanje procesa
- Omogućavaju trapove u softveru (softverski implementirani opkodi)

Precizni izuzeci u protočnoj strukturi

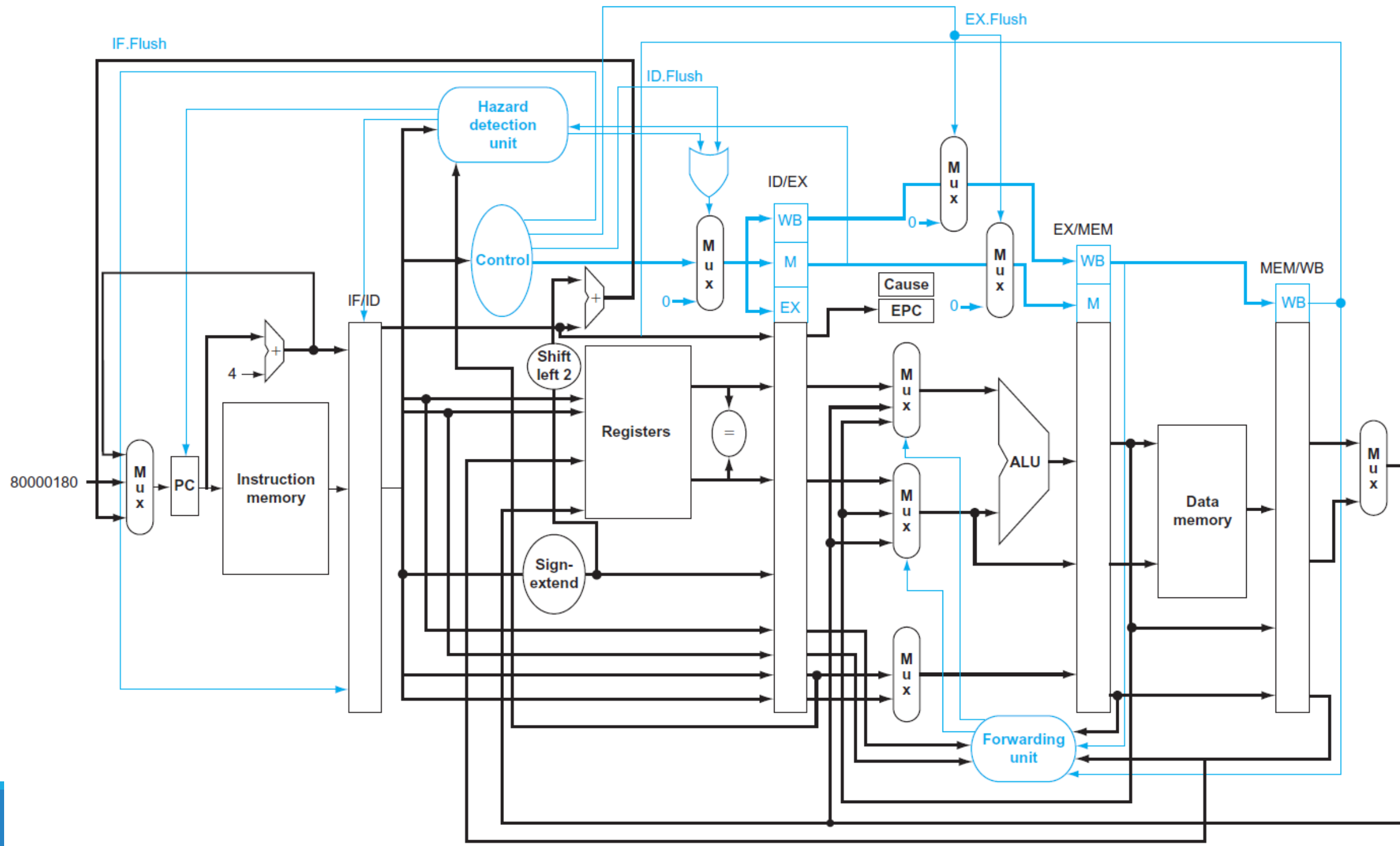
- Kada se desi izuzetak, procesor prekida normalan tok izvršavanja.
- Potrebno je sačuvati adresu instrukcije koja je izazvala izuzetak u registru **EPC (Exception Program Counter)**. **Ovako procesor sačuva mjesto nastanka izuzetka.**
- Kontrola se zatim predaje operativnom sistemu. Pored adrese instrukcije, **OS mora znati i uzrok nastanka izuzetka.** Dva pristupa:

1. Cause registar – ovo koristi MIPS

- On sadrži polje koje kodira razlog izuzetka.
(10 → nedefinisana instrukcija; 12 → aritmetičko prekoračenje)
- Svi izuzeci idu na jednu ulaznu adresu (npr. 0x80000180).
- OS čita Cause registar i na osnovu toga određuje vrstu obrade.

2. Vektorizovani izuzeci

- adresa na koju se prelazi zavisi od vrste izuzetka.
(npr. Nedefinisana instrukcija → 0x80000000; Aritmetičko prekoračenje → 0x80000180)
- OS odmah zna uzrok na osnovu ulazne adrese.



Izuzetak u protočnoj strukturi – primjer

Sekvenca instrukcija:

40 sub \$11, \$2, \$4

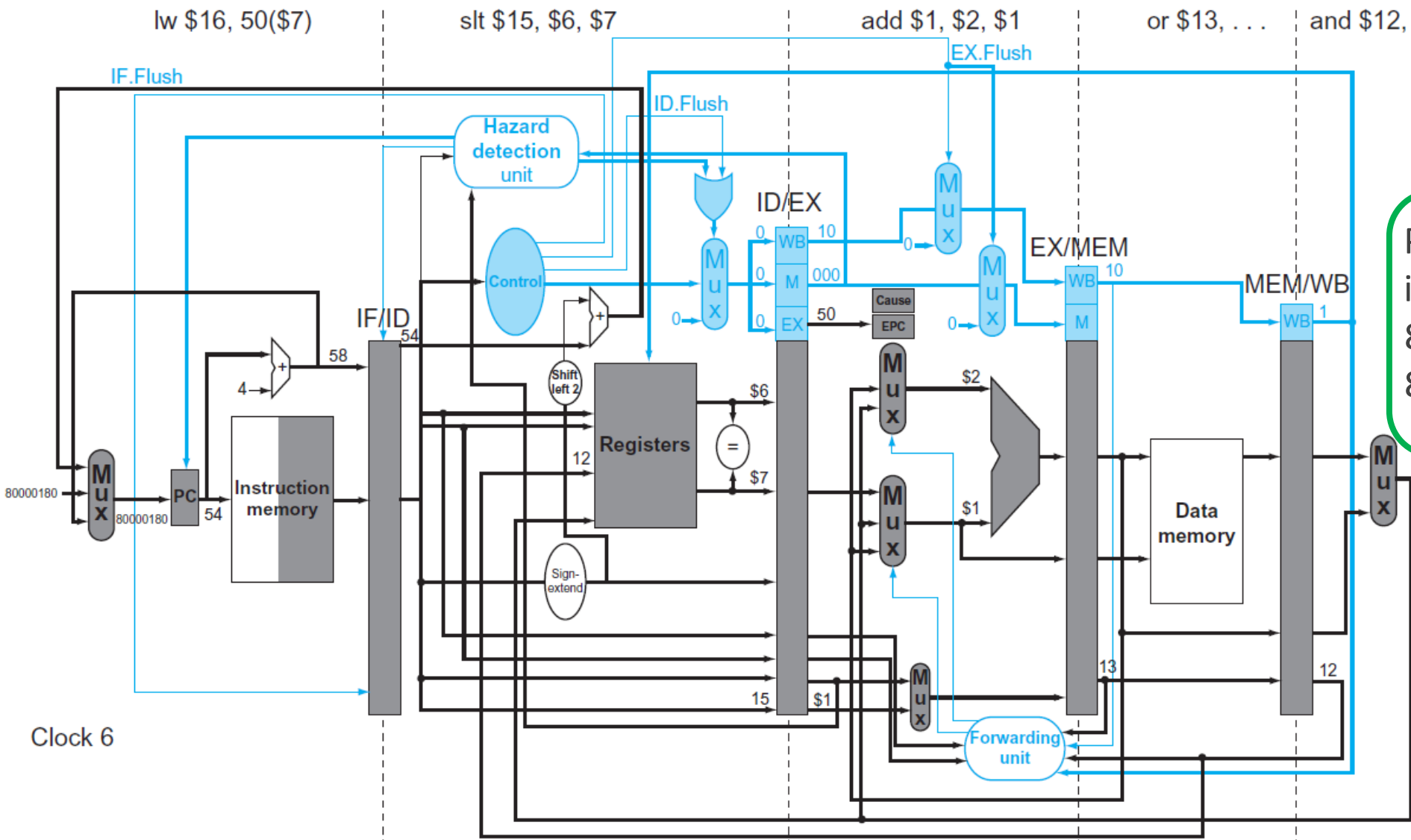
44 and \$12, \$2, \$5

48 or \$13, \$2, \$6

4C add \$1, \$2, \$1

50 slt \$15, \$6, \$7

54 lw \$16, 50(\$7)



Početak rutine za obradu izuzetaka:

80000180 sw \$26,1000(\$0)

80000184 sw \$27,1004(\$0)

...

Šta se dešava u protočnoj strukturi ako se u instrukciji **add** desi aritmetičko prekoračenje?

Izuzetak u protočnoj strukturi – primjer

Sekvenca instrukcija:

40 sub \$11, \$2, \$4

44 and \$12, \$2, \$5

48 or \$13, \$2, \$6

4C add \$1, \$2, \$1

50 slt \$15, \$6, \$7

54 lw \$16, 50(\$7)

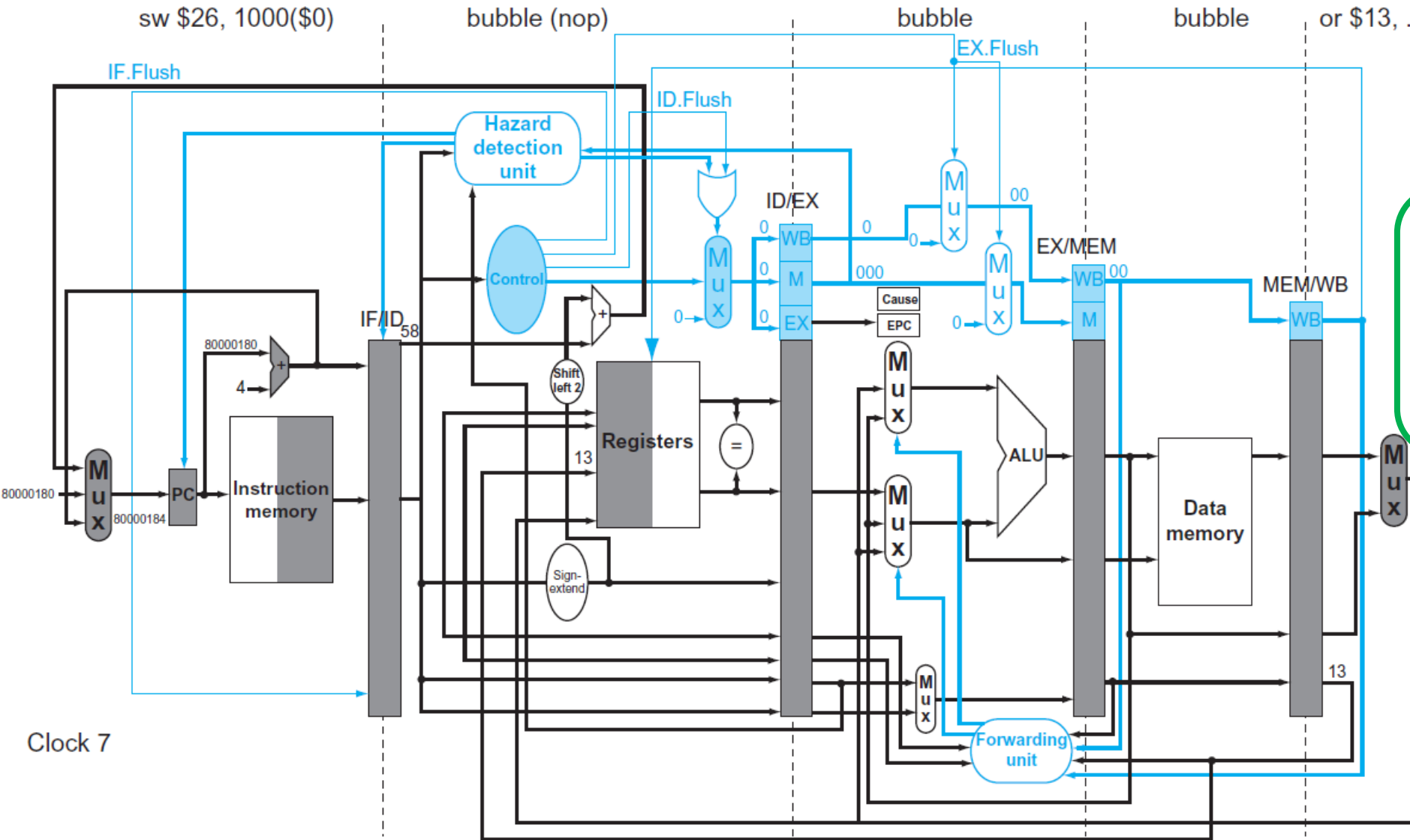
Početak rutine za obradu izuzetaka:

80000180 sw \$26,1000(\$0)

80000184 sw \$27,1004(\$0)

...

Šta se dešava u protočnoj strukturi ako se u instrukciji **add** desi aritmetičko prekoračenje?



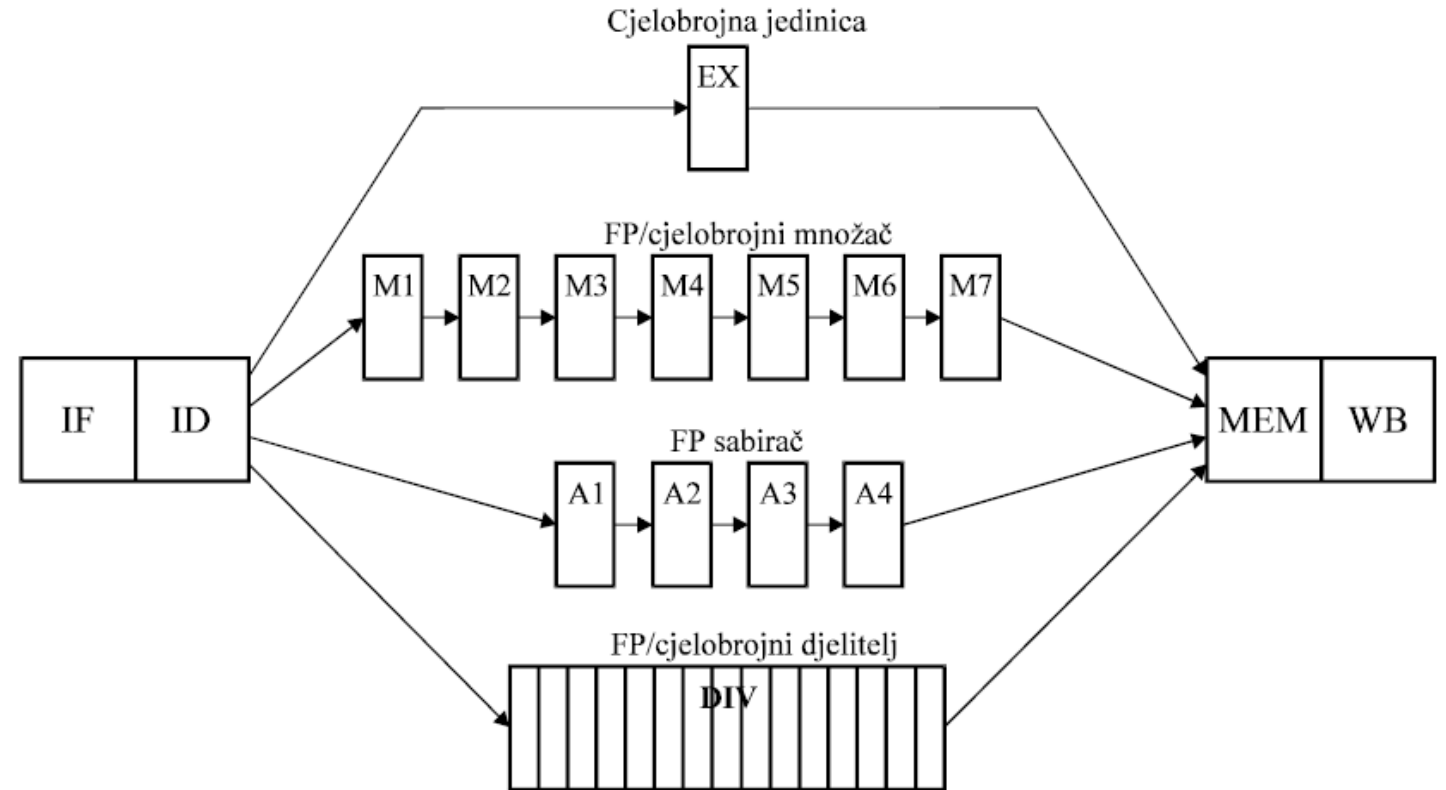
Višeciklusne operacije

Zašto postoje višeciklusne instrukcije?

- ✓ neke operacije su hardverski složenije
- ✓ množenje i dijeljenje su sporiji od sabiranja
- ✓ floating-point operacije imaju veće kašnjenje od cjelobrojnih operacija

Rješenje:

- produžiti trajanje ciklusa tako da i najsporija operacija u EX fazi stigne završiti u jednom ciklusu,
- ili bolje rješenje: **podijeliti EX fazu sporijih instrukcija na više podfaza (više ciklusa)**, tako da se ne mora produžavati ciklus za sve instrukcije. **EX faza može biti protočna, ali ne mora** - zavisno od organizacije izvršne jedinice.




```

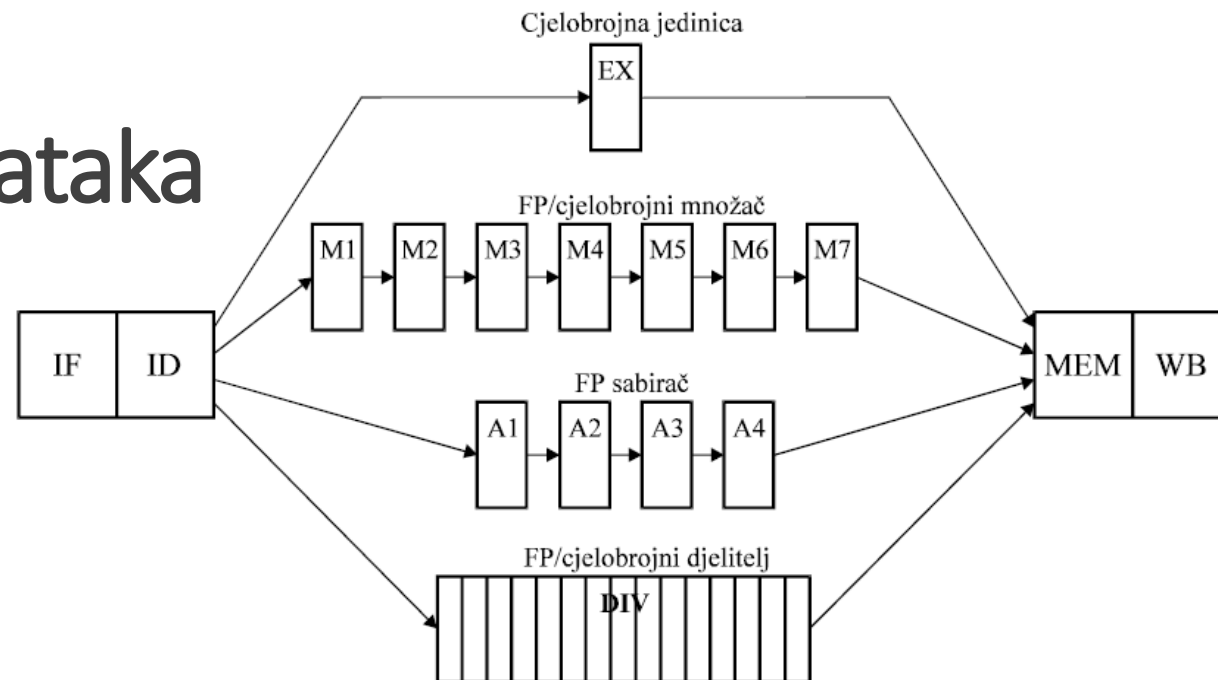
20 sub $5, $2, $4
24 div $6, $1, $5
28 or  $1, $2, $8
2C add $6, $9, $8

```

Tri tipa hazarda podataka

Razlikujemo tri osnovna tipa hazarda podataka:

- 1) RAW - Read After Write**
Stvarna zavisnot (true dependency)
- 2) WAR - Write After Read**
Anti-zavisnost (anti-dependency)
- 3) WAW - Write After Write**
Izlazna zavisnost (output dependency)



U MIPS protočnoj strukturi smo već vidjeli kako se javlja (i rješava: prosljeđivanjem + zastoјima) RAW hazard.

Sa pojavom višeciklusnih operacija može se javiti i WAW hazard!

Instrukcije se više ne završavaju nužno istim redoslijedom kojim su izdate!

Ciklus:	1	2	3	4	5	6	7	8	9	10	11	12
sub \$5, \$2, \$4	IF	ID	EXE	MEM	WB							
div \$6, \$1, \$5		IF	ID	D1	D2	D3	D4	D5	D6	D7	MEM	WB
or \$1, \$2, \$8			IF	ID	EXE	MEM	WB					
add \$6, \$9, \$8				IF	ID	EXE	MEM	WB				

Precizni izuzeci sa višeciklusnim operacijama

20 sub \$5, \$2, \$4
24 div \$6, \$1, \$5
28 or \$1, \$2, \$8
2C add \$6, \$9, \$8

Ideja: Neka svaka operacija u ALU-u traje isto vremena.

Nedostatak: Kašnjenje nasprorije operacije u EXE fazi određuje kašnjenje svih instrukcija.

Šta je sa memorijskim operacijama?

Da li svaka funkcionalna jedinica treba trajati onoliko ciklusa koliko iznosi najgori slučaj?

Zaključak: Može bolje...

Ciklus:	1	2	3	4	5	6	7	8	9	10	11	12
sub \$5, \$2, \$4	IF	ID	EXE	MEM	WB							
div \$6, \$1, \$5		IF	ID	D1	D2	D3	D4	D5	D6	D7	MEM	WB
or \$1, \$2, \$8			IF	ID	EXE	MEM	WB					
add \$6, \$9, \$8				IF	ID	EXE	MEM	WB				

Šta nije u redu sa ovom tabelom u VN arhitekturi?

1. **Sekvencijalna semantika ISA-e nije očuvana!!!**
2. Šta ako instrukcija DIV izazove izuzetak (npr. dijeljenje nulom) u D6 fazi???

Ciklus:	1	2	3	4	5	6	7	8	9	10	11	12	13	14
sub \$5, \$2, \$4	IF	ID	EX1	EX2	EX3	EX4	EX5	EX6	EX7	MEM	WB			
div \$6, \$1, \$5		IF	ID	EX1	EX2	EX3	EX4	EX5	EX6	EX7	MEM	WB		
or \$1, \$2, \$8			IF	ID	EX1	EX2	EX3	EX4	EX5	EX6	EX7	MEM	WB	
add \$6, \$9, \$8				IF	ID	EX1	EX2	EX3	EX4	EX5	EX6	EX7	MEM	WB

Precizni izuzeci sa višeciklusnim operacijama

20 sub \$5, \$2, \$4
24 div \$6, \$1, \$5
28 or \$1, \$2, \$8
2C add \$6, \$9, \$8

Ciklus:	1	2	3	4	5	6	7	8	9	10	11	12
sub \$5, \$2, \$4	IF	ID	EXE	MEM	WB							
div \$6, \$1, \$5		IF	ID	D1	D2	D3	D4	D5	D6	D7	MEM	WB
or \$1, \$2, \$8			IF	ID	EXE	MEM	WB					
add \$6, \$9, \$8				IF	ID	EXE	MEM	WB				

Tehnike:

1. Reorder buffer - bafer promjene redoslijeda
2. History buffer
3. Future register file
4. Checkpointing

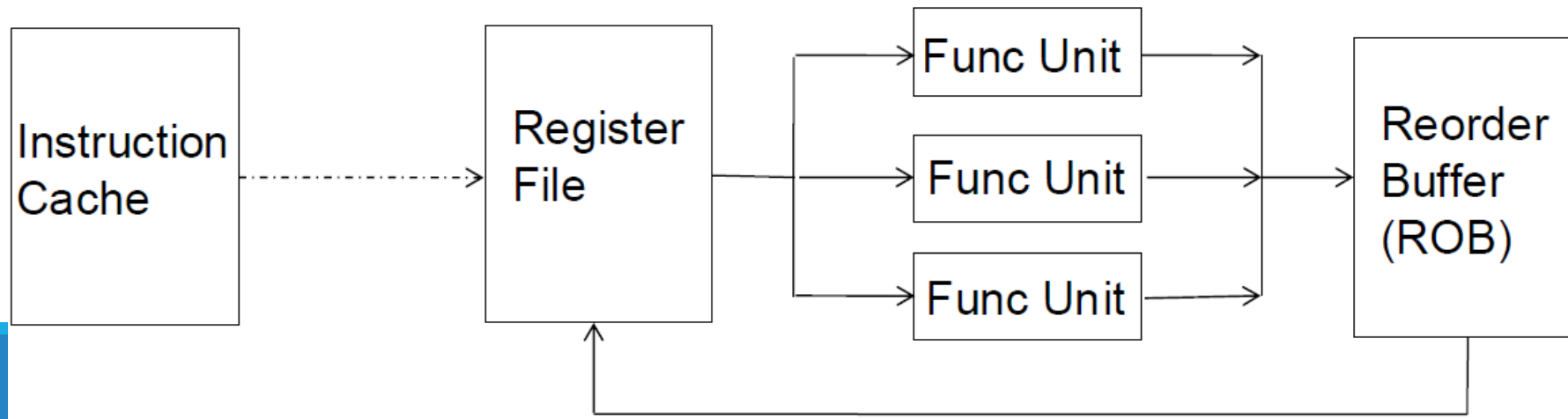
Reorder buffer i precizni izuzeci

20 sub \$5, \$2, \$4
24 div \$6, \$1, \$5
28 or \$1, \$2, \$8
2C add \$6, \$9, \$8

Ciklus:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
sub \$5, \$2, \$4	IF	ID	EXE	MEM	R	W									
div \$6, \$1, \$5		IF	ID	D1	D2	D3	D4	D5	D6	D7	MEM	R	W		
or \$1, \$2, \$8			IF	ID	EXE	MEM	R							W	
add \$6, \$9, \$8				IF	ID	EXE	MEM	R							W

Ideja ROB-a: Završavati instrukcije van redoslijeda, ali ih preurediti prije nego što rezultati postanu vidljivi arhitekturnom stanju.

1. Kada se instrukcija **dekodira**, rezerviše naredni sekvencijalni ulaz u Reorder Buffer-u (ROB)
2. Kada instrukcija **završi**, upisuje rezultat u svoj ulaz u ROB-u (**R faza**)
3. Kada instrukcija postane **najstarija u ROB-u** i **završi bez izuzetaka**, njen rezultat se prenosi u registarsku datoteku ili u memoriju (**W faza**)



Reorder buffer

ROB je hardverska struktura koja čuva informacije o svim instrukcijama koje su dekodirane, ali još nisu završene (committed/retired).

Implementiran je kao cirkularni bafer.

V	Dest RegID	Dest Val	Store Adr	Store Data	PC	Valid bits za reg/data + Ctrl bits	Exception?
---	---------------	-------------	--------------	---------------	----	---------------------------------------	------------

Sadrži sve potrebno da se:

1. instrukcije ispravno vrate u programski redoslijed
2. arhitekturno stanje ažurira rezultatima instrukcije (ako se instrukcija može završiti bez problema)
3. izuzetak ili prekid obradi precizno, (ako je izuzetak ili prekid potrebno obraditi prije nego što se instrukcija završi)
4. Potrebni su biti validnosti da bi se pratila spremnost rezultata i utvrdilo da li je instrukcija završila izvršavanje

Red 0	
Red 1	
Red 2	
...	
Red 15	

Najstarija ins.
(pokazivač na ulaz u ROB-u koji odgovara najstarijoj aktivnoj instrukciji)

Najmlađa ins.

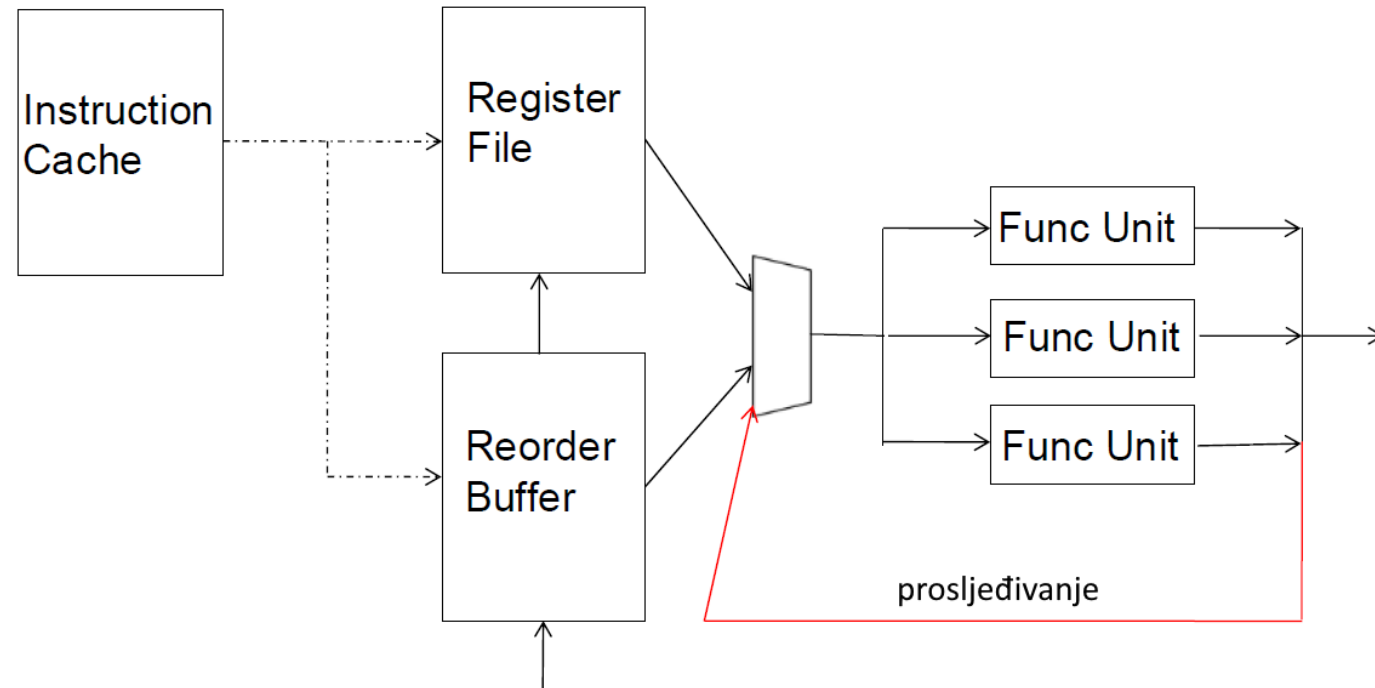
Reorder buffer

- Rezultat se prvo upisuje u ROB kada instrukcija završi izvršavanje.
- Rezultat se upisuje u registarsku datoteku/memoriju u trenutku commit-a.

- **Šta ako nekoj kasnijoj instrukciji treba vrijednost koja se nalazi u reorder buffer-u?**

- Jedna opcija:
zaustaviti operaciju → zaustaviti protočnu strukturu
- **Bolje rješenje:**
pročitati vrijednost iz reorder buffer-a.

Sada se vrijednost registra može se nalaziti u registarskoj datoteci, u reorder buffer-u ili može biti proslijeđena.



Reorder buffer vs. History buffer

Prednosti ROB-a:

1. Konceptualno jednostavno rješenje za podršku preciznim izuzecima
2. Može eliminisati lažne zavisnosti (WAR, WAW)

Nedostaci ROB-a:

1. Reorder buffer-u se mora pristupiti da bi se dobili rezultati koji još nisu upisani u registarsku datoteku

Ideja history buffer-a: **Ažurirati registarsku datoteku kada instrukcija završi, ali PONIŠTITI AŽURIRANJA ako se desi izuzetak.**

1. Kada se instrukcija dekodira, rezerviše ulaz u HB-u.
2. Kada instrukcija završi, stara vrijednost odredišta se smješta u HB.
3. Kada instrukcija postane najstarija i nema izuzetaka, odgovarajući ulaz u HB-u se odbacuje.
4. Kada instrukcija postane najstarija i izuzetak treba obraditi, stare vrijednosti iz HB-a se upisuju nazad u arhitekturno stanje, od repa prema glavi.

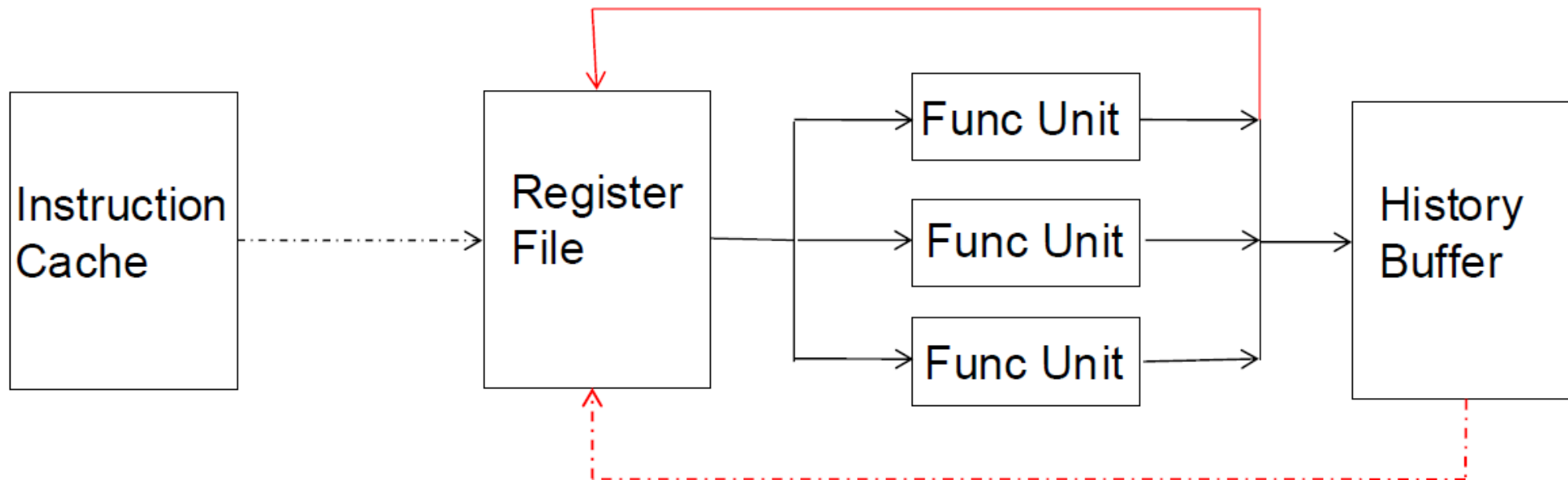
History buffer

Prednosti HB-a:

1. Registarska datoteka sadrži ažurne vrijednosti za dolazeće instrukcije.
(Pristup history buffer-u nije na kritičnom putu)

Nedostaci HB-a:

1. Potrebno je pročitati staru vrijednost odredišnog registra
2. Potrebno je “odmotati” history buffer pri izuzetku
(Povećava se vrijeme potrebno za obradu izuzetaka)



Koristi se samo za izuzetke

Future file

Reorder buffer:

1. Pesimistično ažuriranje registarske datoteke.
2. Ažuriranje samo u programskom redoslijedu.
3. Dovodi do složenosti / kašnjenja pri pristupu novim vrijednostima.

History buffer:

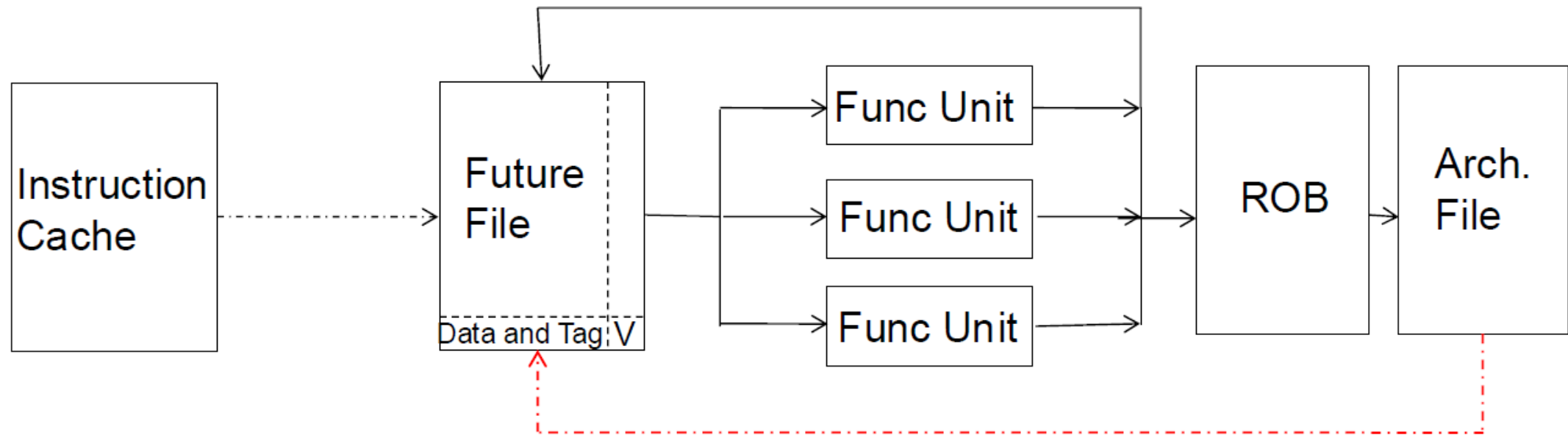
1. Optimistično ažuriranje registarske datoteke.
2. Ažuriranje odmah, ali uz bilježenje stare vrijednosti radi oporavka.
3. Dovodi do složenosti / kašnjenja pri evidentiranju starih vrijednosti.

Ideja: Održavati dvije registarske datoteke (spekulativnu i arhitekturnu)

Arhitekturna registarska datoteka: ažurira se u programskom redoslijedu zbog preciznih izuzetaka. Koristi se reorder buffer kako bi se obezbijedilo ažuriranje redom.

Spekulativna registarska datoteka: ažurira se čim instrukcija završi (ako je ta instrukcija najmlađa koja upisuje taj registar)

Future file



Koristi se samo za izuzetke

Frontend registarska datoteka: Future file se koristi za brz pristup najnovijim vrijednostima registara (spekulativno stanje).

Backend registarska datoteka: Arhitekturna registarska datoteka koristi se za oporavak stanja u slučaju izuzetaka.

Future file

R faza: upisati rezultat u reorder buffer i u spekulativnu registarsku datoteku

W faza: provjeriti da li postoji izuzetak:

- **ako ne postoji:** upisati rezultat u arhitekturnu registarsku datoteku ili u memoriju;
- **ako postoji:** isprazniti protočnu strukturu, kopirati arhitekturnu datoteku u spekulativnu datoteku i nastaviti izvršenje od rutine za obradu izuzetka

Prednost FF-a:

1. Nema potrebe čitati nove vrijednosti iz ROB-a, niti staru vrijednost odredišnog registra

Nedostatak FF-a:

1. Više registarskih datoteka
2. Potrebno je kopirati arhitekturnu registarsku datoteku u spekulativnu datoteku pri izuzetku

Future file i RAT

Možemo li smanjiti trošak dvije registarske datoteke?

Ideja: **koristiti indirekciju (pokazivače) na podatke u frontend i backend dijelu**

1. imati jedno jedinstveno spremište koje čuva vrijednosti registara
2. održavati dvije mape registara (spekulativnu i arhitekturnu). Zovu se: **register alias tables (RATs)**

Frontend RAT: koristi se za brz pristup najnovijim vrijednostima registara (spekulativno stanje)

Retirement/backend RAT: koristi se za oporavak stanja uzrokovan izuzecima (arhitekturno stanje)

NetBurst

